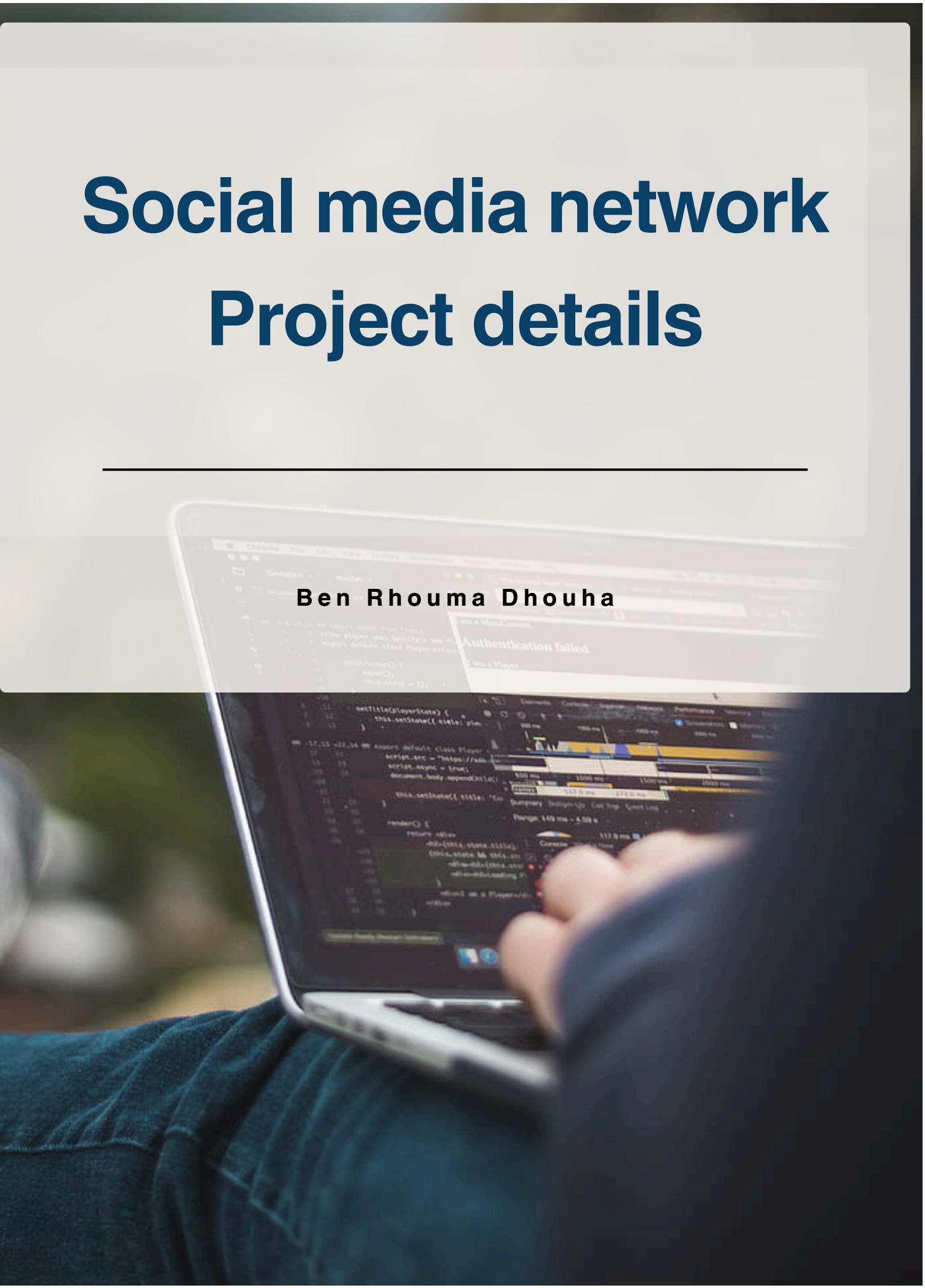
Social media network
Project details

Ben Rhouma Dhouha



SUMMARY

1 PERSONAL PRESENTATION

2 SUMMARY

3 INTRODUCTION

4 DEVELOPMENT ENVIRONMENT

5 IMPLEMENTED FUNCTIONS (1)

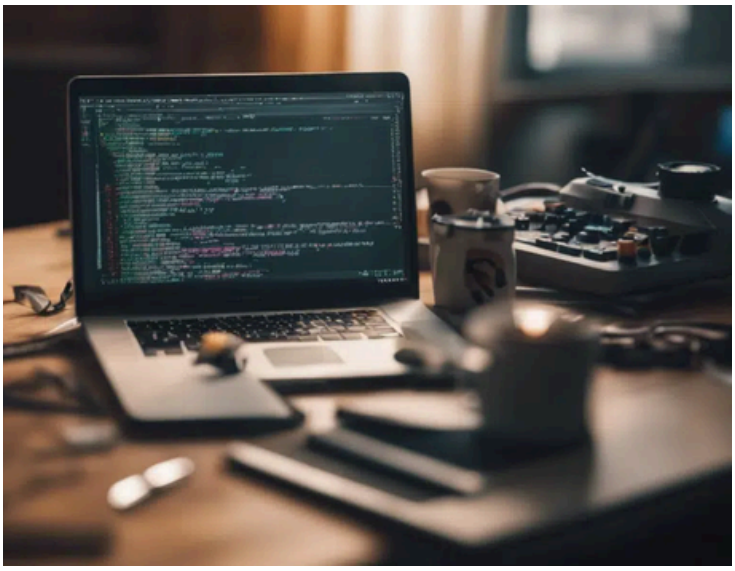
6 IMPLEMENTED FUNCTIONS (2)

7 EXECUTION EXAMPLE

8 CONCLUSION

INTRODUCTION

In an increasingly interconnected world, social networks play a central role in interactions between individuals. Understanding and modeling them therefore become important challenges, both in computer science and in the social sciences.



This project aims to model a social network as a weighted graph using the C programming language. Each user is represented as a vertex, and each relationship as an edge with a weight indicating the strength of the connection.
I implemented various functionalities such as adding users, managing relationships, searching for direct and indirect friends, detecting communities, and calculating optimal paths. This work applies the concepts of abstract data types (ADTs), linked lists, and graph traversal algorithms studied in class.

Development Environment

PROGRAMMING LANGUAGE USED

C language

TOOLS

The project was entirely developed on 64-bit Windows, using:

- Code::Blocks as the development environment.
- Graphviz for the graphical visualization of the graph representing the social network.

PROJECT STRUCTURE AND SOURCE FILES

🔧 Header and Implementation Files

- GRAPHEstruct.h: definition of the graph structure.
- graphe.h / graphe.c: declaration and implementation of functions related to graph management.
- ELTstruct.h: structure of users (ELEMENT).
- ELTUSER.h / ELTUSER.c: management of operations on users.

Data Files

- reseau.txt: file containing users and their relationships.
- graphe.dot: file automatically generated for graph visualization in DOT format (used by Graphviz).
- graphe.png: image generated from the DOT file to graphically represent the social network.

Implemented Functions

1- Addition and Deletion

My project mainly relies on multiple and diverse functionalities that were extremely helpful to me. I cite one of these functions, which is almost the most important one: the “Add User” function. Before adding any user, it checks whether the user already exists and displays an indication about the possibility of adding them.

Several other functions are implemented in a similar way, such as Delete User, Add Relationship, and others.

```
int ajouterUtilisateur(grapheOrdonne g, ELEMENT user) //1
{
    if (!g)
    {
        printf("!g\n");
        return 0;
    }

    if (!user)
    {
        printf("!user\n");
        return 0;
    }

    if (user->id < 0)
    {
        printf("user->id < 0\n");
        return 0;
    }

    // Vérifie si l'utilisateur existe déjà
    if (!unique(ids, g->v, user->id))
    {
        printf("L'utilisateur existe déjà !!!\n");
        return 0;
    }

    // Tentative de réallocation mémoire
    ELEMENT* tmpUtilisateurs = realloc(g->utilisateurs, (g->v + 1) * sizeof(ELEMENT));
    if (!tmpUtilisateurs)
    {
        printf("Erreur de réallocation utilisateurs\n");
        return 0;
    }

    Node* tmpAdjList = realloc(g->adjList, (g->v + 1) * sizeof(Node));
    if (!tmpAdjList)
    {
        printf("Erreur de réallocation adjList\n");
        return 0;
    }

    int* tmpIds = realloc(ids, (g->v + 1) * sizeof(int));
    if (!tmpIds)
    {
        printf("Erreur de réallocation ids\n");
        free(tmpUtilisateurs);  // Libère mémoire allouée pour tmpUtilisateurs
        free(tmpAdjList);
        return 0;
    }

    // Mise à jour des pointeurs
    g->utilisateurs = tmpUtilisateurs;
    g->adjList = tmpAdjList;
    ids = tmpIds;

    // Ajout des valeurs
    g->utilisateurs[g->v] = user;
    g->adjList[g->v] = NULL;  // Important : initialiser la liste à NULL
    ids[g->v] = user->id;

    g->v++;

    printf("Utilisateur [ID:%d] %s ajouté (position %d)\n",
           user->id, user->nom, g->v - 1);

    return 1;
```

REQUIRED CONTROL

Since g->utilisateurs, g->adjListe, and ids (a global variable declared in graphe.c to provide more flexibility in choosing identifiers) are dynamic arrays initially created using malloc with v users, we use realloc in order to better ensure flexibility in the number of users.

Fonctions implémentées

2- Traversal Functions:

These functions mainly play the role of graph traversal between the different nodes/users, whether to determine the shortest paths or to search for relationships: DFS, BFS, and Dijkstra.

3- Find Community:

This function identifies communities (connected components) of a graph using DFS.
Each community is numbered and displayed in color in the terminal.
A visite[] array marks the vertices that have already been explored.
Users belonging to the same community are listed with their name and ID.
Everything is clearly visualized with proper indentation and color coding.

4- Display Functions:

- isGraphvizInstalled checks the presence of Graphviz using the system command dot -v.
- It returns 1 if Graphviz is available, and 0 otherwise.
- generateAndOpenImage converts a .dot file into a .png image using dot.
- If the generation is successful, the image is automatically opened (Windows/Linux).
- In case of failure, error messages guide the user (Graphviz availability, file validity).

These functions make it easier to automatically visualize graphs directly from the program.
Note: If you want to run the program on your machine and do not have Graphviz installed, that is not a problem. You can simply replace the Graphviz-related functions with the function implemented in graphe.c, afficherGraphe.

Execution Example

1

RESEAU.TXT

reseau.txt - Notepad

File Edit Format View Help

```
U 0 ahmed 10
U 1 ines 12
U 2 mohamed 8
U 3 hana 20
U 4 sami 17
U 7 dhouha 25
U 773 Abir 23
U 50 ISGT 300
R 0 1 55
R 2 3 4
R 2 8 10
R 7 8 9
R 7 773 55
R 7 50 800
```

We have these initializations

2

ADD A RELATIONSHIP BETWEEN DHOUHA AND MOHAMED

```
=====================================
ID source : 7
ID destination : 2
poids : 20

Appuyez sur Entrée pour continuer...
```

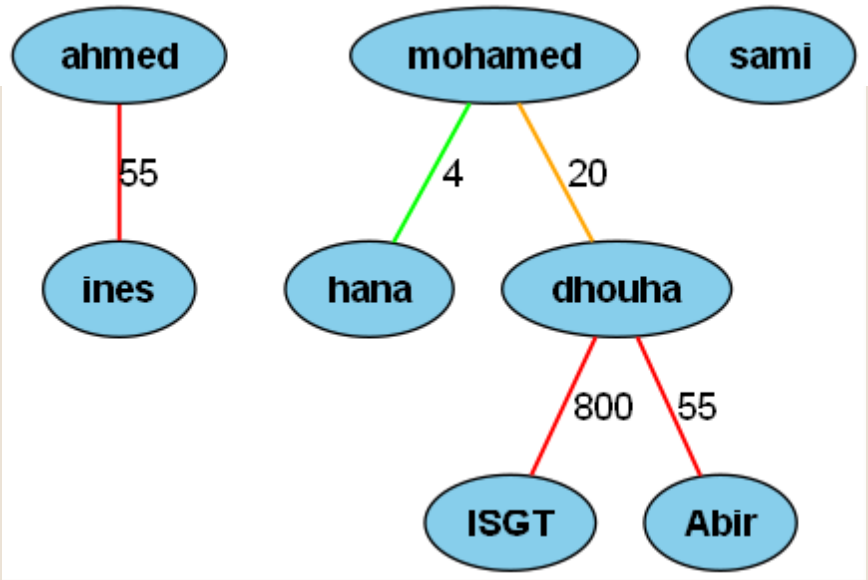
HERE IS THE MAIN MENU:

```
 1   - Charger le graphe depuis le fichier -reseau.txt-
 2   - Afficher le graphe
 3   - Ajouter un utilisateur
 4   - Supprimer un utilisateur
 5   - Ajouter une relation
 6   - Supprimer une relation
 7   - Trouver les amis (DFS)
 8   - Trouver les amis à une distance N (BFS)
 9   - Trouver le chemin le plus fort (Dijkstra)
10   - Détecter les communautés
11   - Sauvegarder
 0   - Quitter

- Choix :
```

3

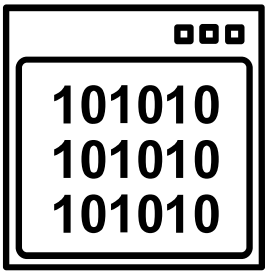
APPLICATION OF THE FIND COMMUNITY FUNCTION

```
Communauté 1 :
  └ ahmed (ID 0)
  └ ines (ID 1)

Communauté 2 :
  └ mohamed (ID 2)
  └ hana (ID 3)
  └ dhouha (ID 7)
  └ Abir (ID 773)
  └ ISGT (ID 50)

Communauté 3 :
  └ sami (ID 4)

Appuyez sur Entrée pour continuer...
```



CONCLUSION

This project was an excellent opportunity to practically apply the concepts like data structures, linked lists, and graph algorithms. By modeling a social network as a weighted graph, I was able to implement various functionalities such as user and relationship management, DFS and BFS traversals, and community detection.

This work also allowed me to **strengthen my skills in C programming**, **code organization and dynamic memory management**, **develop my research skills**, and **create more polished interfaces and graph visualizations**.

It provides a solid foundation for tackling more complex projects, especially in the field of network analysis.